

From Variational Classifiers to Linear Classifiers

QML

Contents

1	Overview	3
2	Classical Classifiers Recap	3
2.1	Linear Classifier	3
2.2	Logistic Regression	3
2.3	Support Vector Machine (SVM)	4
2.4	From Features to Decision Boundaries	4
3	Variational Quantum Classifier (VQC)	4
3.1	The Big Picture	4
3.2	Stage 1: Feature Map (Encoding)	5
3.3	Stage 2: Ansatz (Trainable Circuit)	6
3.4	Stage 3: Measurement	6
3.5	The Full VQC Circuit	7
4	How VQC Works as a Classifier	7
4.1	The Classification Pipeline	7
4.2	Forward Pass in Detail	8
4.3	Encoding Data: What Happens Geometrically	8
4.4	The Ansatz: Learning a Rotation	8
4.5	Parameterized Layers and Expressibility	8
5	Cost Functions for Classification	9
5.1	Parity-Based Cost	9
5.2	Cross-Entropy Loss (Adapted)	9
5.3	Hinge Loss (SVM-style)	9
5.4	Multi-Class Extension	10
6	The Classical Optimization Loop	10
6.1	How Optimization Works	10
6.2	Gradient Computation	10
6.3	Common Optimizers for VQC	11

7	From Variational to Linear: The Key Insight	11
7.1	The Surprising Connection	11
7.2	Making It Explicit	11
7.3	The Implications	12
7.4	Proof Sketch	12
7.5	Connection to Kernel Methods	12
8	When Does the Variational Approach Help?	13
8.1	VQC vs Direct Kernel Method	13
8.2	When VQC Goes Beyond Kernel	13
9	Limitations of the Variational Approach	14
9.1	Local Minima	14
9.2	Shot Noise	14
9.3	Circuit Depth	14
9.4	Barren Plateaus (Preview)	14
10	Practical Considerations	15
10.1	Choosing the Feature Map	15
10.2	Choosing the Ansatz	15
10.3	Choosing the Optimizer	15
10.4	Hyperparameter Guidelines	15
11	Example: VQC on Iris Dataset	15
11.1	Setup	15
11.2	Expected Results	16
11.3	The Qiskit Code Flow	16
12	Summary	17

1 Overview

this lecture is where QML gets real. we move from the abstract PQC framework to actually building quantum classifiers that take classical data and output predictions. the variational quantum classifier (VQC) is the poster child of NISQ QML — but understanding its structure deeply reveals smtg surprising: many VQCs r secretly just linear classifiers in a quantum feature space. this connection to kernel methods (lecture 12) is one of the deepest results in QML theory.

we'll start by recapping classical classifiers, then build up the VQC architecture piece by piece, analyze how it works as a classifier, and finally show the surprising reduction to linear classification.

2 Classical Classifiers Recap

2.1 Linear Classifier

the simplest classifier: a hyperplane that separates two classes.

Definition 2.1 (Linear Classifier). given input $x \in \mathbb{R}^d$, a linear classifier computes:

$$f(x) = \text{sign}(w^\top x + b)$$

where $w \in \mathbb{R}^d$ is the weight vector and $b \in \mathbb{R}$ is the bias. the decision boundary is the hyperplane $\{x : w^\top x + b = 0\}$.

Example 2.1. for $d = 2$, $w = (1, -1)^\top$, $b = 0$:

$$f(x_1, x_2) = \text{sign}(x_1 - x_2)$$

this classifies points above the line $x_1 = x_2$ as $+1$ and points below as -1 . simple, fast, interpretable — but can only separate linearly separable data.

2.2 Logistic Regression

instead of a hard sign, use a soft sigmoid:

Definition 2.2 (Logistic Regression).

$$P(y = 1|x) = \sigma(w^\top x + b) = \frac{1}{1 + e^{-(w^\top x + b)}}$$

the output is a probability. train by maximizing log-likelihood (equivalently, minimizing cross-entropy loss):

$$\mathcal{L}(w, b) = -\frac{1}{N} \sum_{i=1}^N [y_i \log \hat{y}_i + (1 - y_i) \log(1 - \hat{y}_i)]$$

where $\hat{y}_i = \sigma(w^\top x_i + b)$.

logistic regression is still a linear model — the decision boundary $w^\top x + b = 0$ is still a hyperplane. but the probabilistic output is much more useful in practice.

2.3 Support Vector Machine (SVM)

SVM finds the maximum-margin separating hyperplane:

Definition 2.3 (Hard-Margin SVM).

$$\min_{w,b} \frac{1}{2} \|w\|^2 \quad \text{subject to} \quad y_i(w^\top x_i + b) \geq 1 \quad \forall i$$

the margin is $2/\|w\|$. maximizing margin = minimizing $\|w\|$.

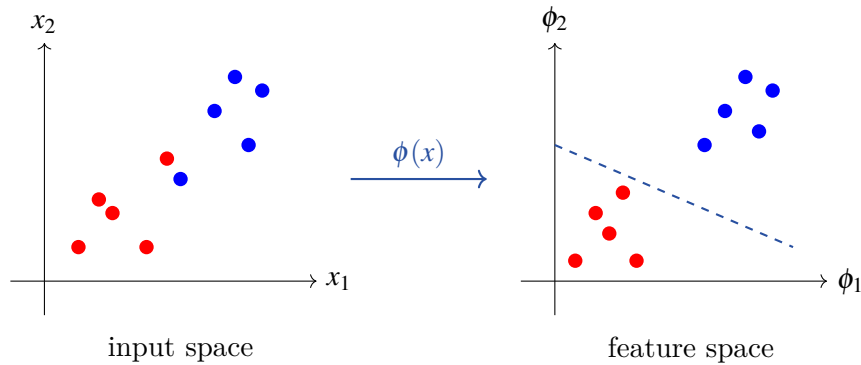
the dual formulation introduces the kernel trick:

$$\max_{\alpha} \sum_i \alpha_i - \frac{1}{2} \sum_{i,j} \alpha_i \alpha_j y_i y_j k(x_i, x_j)$$

where $k(x_i, x_j) = \phi(x_i)^\top \phi(x_j)$ is the kernel function. this lets SVMs work in high-dimensional feature spaces without explicitly computing the features.

Key Point. all three classical classifiers are fundamentally linear — they separate data with hyperplanes. the SVM kernel trick is the classical version of what quantum feature maps do. keep this connection in mind as we build VQCs.

2.4 From Features to Decision Boundaries

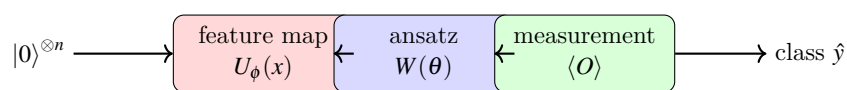


the whole point of feature maps: make data linearly separable in a higher-dimensional space. quantum circuits do this naturally because the Hilbert space is exponentially large.

3 Variational Quantum Classifier (VQC)

3.1 The Big Picture

a VQC has three stages:



1. feature map $U_\phi(x)$: encodes classical data x into a quantum state
2. ansatz $W(\theta)$: trainable parameterized circuit

3. measurement: extract a classical prediction from the quantum state

the full unitary is $U(x, \theta) = W(\theta)U_\phi(x)$, and the output is:

$$f(x; \theta) = \langle 0 |^{\otimes n} U_\phi^\dagger(x) W^\dagger(\theta) O W(\theta) U_\phi(x) | 0 \rangle^{\otimes n}$$

where O is the measurement observable (e.g., Z_0 on the first qubit).

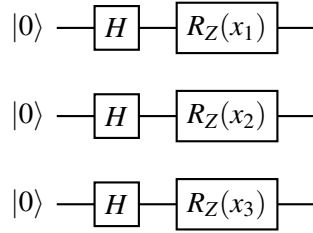
3.2 Stage 1: Feature Map (Encoding)

the feature map encodes classical data into a quantum state. we've seen these in lecture 09, but let's be specific about the ones used in VQCs.

ZFeatureMap:

$$U_\phi(x) = \prod_{\ell=1}^r \left[\bigotimes_{i=1}^n R_Z(x_i) \cdot H^{\otimes n} \right]$$

for 1 repetition ($r = 1$), on 3 qubits:



this creates the state:

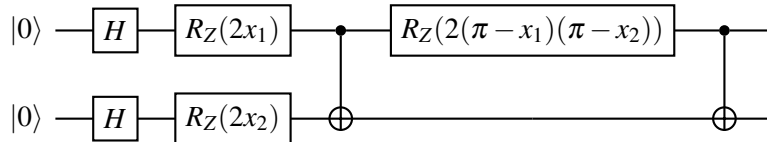
$$|\phi(x)\rangle = \bigotimes_{i=1}^n \frac{1}{\sqrt{2}} (|0\rangle + e^{ix_i} |1\rangle)$$

each qubit encodes one feature as a phase. no entanglement $\Rightarrow$ tensor product structure.

ZZFeatureMap:

$$U_\phi(x) = \prod_{\ell=1}^r \left[\prod_{i < j} e^{i(\pi - x_i)(\pi - x_j)Z_i Z_j} \cdot \prod_i R_Z(2x_i) \cdot H^{\otimes n} \right]$$

for 2 qubits, 1 rep:



the ZZ interaction creates entanglement between qubits, encoding pairwise feature correlations. this is the encoding from havlíček et al. (2019) — the one that led to the quantum kernel method.

Key Point. the choice of feature map determines the quantum feature space. ZFeatureMap gives a tensor product (unentangled) feature space. ZZFeatureMap gives an entangled feature space that can encode feature interactions. the entangled version is much harder to simulate classically.

PauliFeatureMap (general):

$$U_\phi(x) = \prod_{\ell=1}^r \left[\exp \left(i \sum_{S \subseteq [n]} \phi_S(x) \prod_{j \in S} P_j \right) \cdot H^{\otimes n} \right]$$

where $P_j \in \{I, X, Y, Z\}$ and $\phi_S(x) = \prod_{j \in S} x_j$ encodes feature products. ZFeatureMap is the special case with $|S| = 1$ and $P = Z$. ZZFeatureMap adds $|S| = 2$ terms.

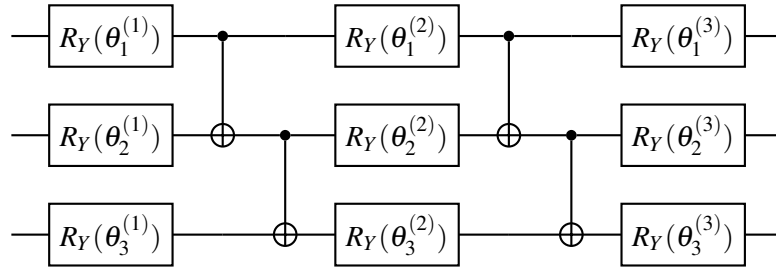
3.3 Stage 2: Ansatz (Trainable Circuit)

the ansatz is the parameterized part we optimize. two main choices in qiskit:

RealAmplitudes ansatz:

$$W(\theta) = \prod_{\ell=1}^L \left[\text{CNOT layer} \cdot \bigotimes_{i=1}^n R_Y(\theta_i^{(\ell)}) \right]$$

for 3 qubits, 2 layers:



called “RealAmplitudes” bcs R_Y rotations only produce real amplitudes (no complex phases). this is the go-to ansatz for many VQC implementations.

parameter count: $n \times (L + 1)$ for n qubits and L entangling layers.

EfficientSU2 ansatz:

$$W(\theta) = \prod_{\ell=1}^L \left[\text{CNOT layer} \cdot \bigotimes_{i=1}^n R_Y(\theta_{i,1}^{(\ell)}) R_Z(\theta_{i,2}^{(\ell)}) \right]$$

adds R_Z rotations for complex amplitudes. more expressive but more parameters: $2n \times (L + 1)$.

3.4 Stage 3: Measurement

we need to extract a classical prediction from the quantum state. two approaches:

approach 1: estimator primitive (expectation value)

measure the expectation value of an observable:

$$\hat{y}(x) = \langle \psi(x, \theta) | O | \psi(x, \theta) \rangle$$

common choice: $O = Z_0$ (pauli Z on first qubit), giving $\hat{y} \in [-1, 1]$.

for binary classification: predict class 0 if $\hat{y} > 0$, class 1 if $\hat{y} \leq 0$.

for probabilistic output: map to $[0, 1]$ via $P(y = 1) = \frac{1 - \hat{y}}{2}$.

approach 2: sampler primitive (probabilities)

measure all qubits in computational basis. classify based on measurement outcome probabilities:

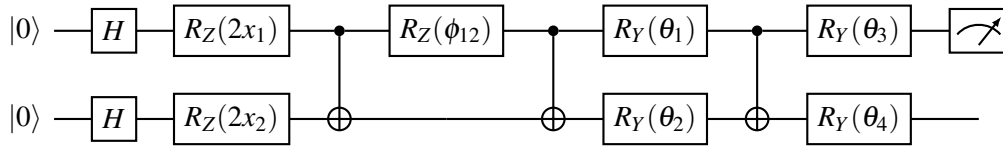
$$P(y = k) = \sum_{z: \text{parity}(z)=k} |\langle z | \psi(x, \theta) \rangle|^2$$

or more simply: predict class 0 if the first qubit measures $|0\rangle$ more often than $|1\rangle$.

Key Point. the estimator primitive gives $\langle O \rangle$ directly (efficient, one number). the sampler primitive gives the full probability distribution (more info, but u need to process it). for binary classification, the estimator with $O = Z_0$ is usually sufficient.

3.5 The Full VQC Circuit

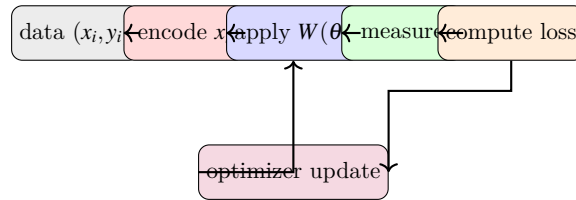
putting it all together — ZZFeatureMap (2 qubits, 1 rep) + RealAmplitudes (1 layer):



where $\phi_{12} = 2(\pi - x_1)(\pi - x_2)$. total: 4 trainable parameters $(\theta_1, \dots, \theta_4)$, 2 data-dependent parameters (x_1, x_2) .

4 How VQC Works as a Classifier

4.1 The Classification Pipeline



repeat until convergence

step by step:

1. take a training sample (x_i, y_i)
2. encode x_i into the quantum state via $U_\phi(x_i) |0\rangle^{\otimes n}$
3. apply the trainable ansatz $W(\theta)$
4. measure to get prediction $\hat{y}_i$
5. compute the loss $\mathcal{L}(\hat{y}_i, y_i)$
6. use a classical optimizer to update θ
7. repeat over the training set until convergence

4.2 Forward Pass in Detail

for input x :

$$|\psi_0\rangle = |0\rangle^{\otimes n} \quad (\text{initial state}) \quad (1)$$

$$|\psi_1\rangle = U_\phi(x) |\psi_0\rangle \quad (\text{encoded state}) \quad (2)$$

$$|\psi_2\rangle = W(\theta) |\psi_1\rangle \quad (\text{after ansatz}) \quad (3)$$

$$\hat{y} = \langle \psi_2 | O | \psi_2 \rangle \quad (\text{measurement}) \quad (4)$$

expanding:

$$\hat{y}(x; \theta) = \langle 0 |^{\otimes n} U_\phi^\dagger(x) W^\dagger(\theta) O W(\theta) U_\phi(x) | 0 \rangle^{\otimes n}$$

this is a function $f: \mathbb{R}^d \times \mathbb{R}^p \rightarrow [-1, 1]$ where d = data dimension, p = number of parameters.

4.3 Encoding Data: What Happens Geometrically

when we encode x via $U_\phi(x) |0\rangle^{\otimes n}$, we map x to a point on the unit sphere in $\mathbb{C}^{2^n}$ (or equivalently, a point on the bloch hypersphere for $n > 1$).

Example 4.1. for 1 qubit with $U_\phi(x) = R_Y(x)$:

$$|\phi(x)\rangle = \cos \frac{x}{2} |0\rangle + \sin \frac{x}{2} |1\rangle$$

this traces out a great circle on the bloch sphere as x varies. the encoding maps the 1D input to a curve on a 2D sphere.

for 2 qubits with tensor product encoding:

$$|\phi(x_1, x_2)\rangle = R_Y(x_1) |0\rangle \otimes R_Y(x_2) |0\rangle$$

the feature space is $\mathbb{C}^4$ (a 4-dimensional state). we go from 2D classical to effectively 4D quantum.

4.4 The Ansatz: Learning a Rotation

the ansatz $W(\theta)$ is a parameterized unitary. geometrically, it rotates the encoded state in hilbert space. training θ = finding the rotation that aligns same-class states in one direction and different-class states in another.

Intuition. think of it this way: the feature map puts data points on a sphere. the ansatz rotates the sphere so that a fixed measurement axis (say, Z) separates the classes. the optimizer finds the rotation angles θ that achieve the best separation.

4.5 Parameterized Layers and Expressibility

more layers in the ansatz = more expressive model:

- 0 layers (identity ansatz): the classifier is completely determined by the feature map + measurement. no learnable parameters.
- 1 layer: can learn simple rotations. equivalent to a linear model in feature space (well prove this later).

- L layers: increasing expressibility. the circuit can represent more complex decision boundaries.
- $\sim n$ layers: the circuit becomes approximately universal — can represent any unitary.

but theres a catch: more layers $\Rightarrow$ more parameters $\Rightarrow$ harder optimization (barren plateaus, lecture 16).

5 Cost Functions for Classification

5.1 Parity-Based Cost

the simplest approach: use the expectation value of Z_0 directly as a loss:

$$\mathcal{L}(\theta) = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i(\theta))^2$$

where $y_i \in \{-1, +1\}$ and $\hat{y}_i = \langle Z_0 \rangle_{x_i, \theta}$.

this is just MSE loss on the quantum output. simple but works.

5.2 Cross-Entropy Loss (Adapted)

for probabilistic classification, map the quantum output to probabilities:

$$p_i = P(y_i = 1 | x_i, \theta) = \frac{1 - \langle Z_0 \rangle_{x_i, \theta}}{2}$$

then use binary cross-entropy:

$$\mathcal{L}(\theta) = -\frac{1}{N} \sum_{i=1}^N [\tilde{y}_i \log p_i + (1 - \tilde{y}_i) \log(1 - p_i)]$$

where $\tilde{y}_i \in \{0, 1\}$ (relabeled from $\{-1, +1\}$).

Key Point. cross-entropy loss has steeper gradients than MSE when predictions r confident but wrong. this often leads to faster convergence for VQCs. in qiskit, the `NeuralNetworkClassifier` supports both.

5.3 Hinge Loss (SVM-style)

adapted from SVMs:

$$\mathcal{L}(\theta) = \frac{1}{N} \sum_{i=1}^N \max(0, 1 - y_i \cdot \hat{y}_i(\theta))$$

penalizes predictions that r on the wrong side of the margin. zero loss when $y_i \cdot \hat{y}_i \geq 1$, i.e., when the prediction is correct with margin.

5.4 Multi-Class Extension

for $C > 2$ classes, use C measurement qubits or C observables:

$$\hat{y}(x) = \arg \max_{c \in \{0, \dots, C-1\}} \langle O_c \rangle_{x, \theta}$$

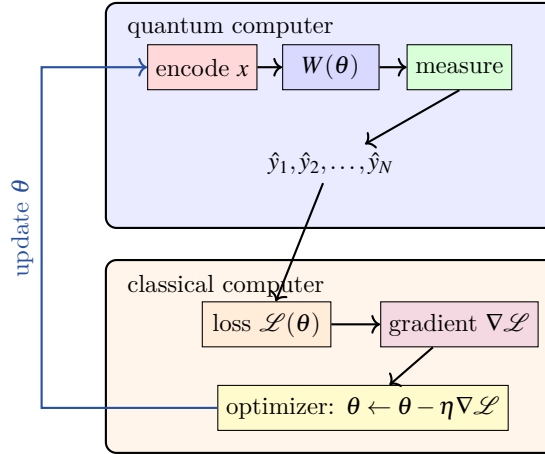
where $O_c = Z_c$ (measure the c -th qubit). combine with softmax cross-entropy:

$$P(y = c|x) = \frac{e^{\langle O_c \rangle}}{\sum_{c'} e^{\langle O_{c'} \rangle}}$$

6 The Classical Optimization Loop

6.1 How Optimization Works

the optimizer is entirely classical. it receives the loss value (and possibly gradients) from the quantum circuit and updates parameters.



6.2 Gradient Computation

two main approaches:

parameter shift rule:

$$\frac{\partial \mathcal{L}}{\partial \theta_k} = \frac{\mathcal{L}(\theta_k + \pi/2) - \mathcal{L}(\theta_k - \pi/2)}{2}$$

requires 2 circuit evaluations per parameter. for p parameters: $2p$ circuits per gradient step.

finite difference:

$$\frac{\partial \mathcal{L}}{\partial \theta_k} \approx \frac{\mathcal{L}(\theta_k + \epsilon) - \mathcal{L}(\theta_k - \epsilon)}{2\epsilon}$$

less accurate (approximation), but same cost. the parameter shift rule is exact for gates of the form $e^{-i\theta G/2}$ where $G^2 = I$.

gradient-free optimizers: COBYLA, Nelder-Mead, SPSA. dont need gradients at all. SPSA is particularly useful bcs it estimates the gradient with only 2 circuit evaluations (regardless of p):

$$\hat{g}_k = \frac{\mathcal{L}(\theta + c\Delta) - \mathcal{L}(\theta - c\Delta)}{2c\Delta_k}$$

where Δ is a random perturbation vector.

6.3 Common Optimizers for VQC

optimizer	gradient?	circuits/step	notes
COBYLA	no	1–few	simple, often good enough
SPSA	stochastic	2	great for noisy hardware
Adam	yes	$2p + 1$	fast convergence, needs gradients
L-BFGS-B	yes	$2p + 1$	good for simulators
NFT	no	$O(p)$	designed for quantum circuits

Key Point. on real hardware, use SPSA or COBYLA — they handle shot noise well. on simulators, use Adam or L-BFGS-B for faster convergence. the parameter shift rule is mathematically exact but expensive ($2p$ circuits per step).

7 From Variational to Linear: The Key Insight

7.1 The Surprising Connection

heres the deep result. consider a VQC with:

- feature map $U_\phi(x)$
- ansatz $W(\theta)$
- observable O

the output is:

$$f(x; \theta) = \langle 0 | U_\phi^\dagger(x) W^\dagger(\theta) O W(\theta) U_\phi(x) | 0 \rangle$$

define the “rotated observable”:

$$\tilde{O}(\theta) = W^\dagger(\theta) O W(\theta)$$

this is a fixed hermitian operator (for given θ). now:

$$f(x; \theta) = \langle 0 | U_\phi^\dagger(x) \tilde{O}(\theta) U_\phi(x) | 0 \rangle = \text{Tr} [\tilde{O}(\theta) \cdot \rho(x)]$$

where $\rho(x) = U_\phi(x) | 0 \rangle \langle 0 | U_\phi^\dagger(x)$ is the encoded density matrix.

Key Point. the VQC output is a linear function of the density matrix $\rho(x)$! the density matrix is the feature vector, and $\tilde{O}(\theta)$ plays the role of the weight matrix. the ansatz parameters θ only determine the weight matrix — they dont change the features.

7.2 Making It Explicit

expand $\tilde{O}(\theta)$ in the pauli basis:

$$\tilde{O}(\theta) = \sum_{\alpha} w_{\alpha}(\theta) P_{\alpha}$$

where $P_{\alpha} \in \{I, X, Y, Z\}^{\otimes n}$ are the 4^n pauli strings and $w_{\alpha}(\theta)$ are real coefficients determined by θ .

then:

$$f(x; \theta) = \sum_{\alpha} w_{\alpha}(\theta) \cdot \text{Tr}[P_{\alpha} \rho(x)] = \sum_{\alpha} w_{\alpha}(\theta) \cdot \phi_{\alpha}(x)$$

where $\phi_{\alpha}(x) = \text{Tr}[P_{\alpha} \rho(x)]$ are the quantum features (expectation values of pauli operators on the encoded state).

this is literally a linear model:

$$f(x; \theta) = w(\theta)^{\top} \phi(x)$$

with feature vector $\phi(x) \in \mathbb{R}^{4^n}$ and weight vector $w(\theta) \in \mathbb{R}^{4^n}$.

7.3 The Implications

1. the VQC is a linear classifier in the quantum feature space, regardless of circuit depth
2. nonlinearity comes from the feature map $\phi(x) = \text{Tr}[P_{\alpha} \rho(x)]$, not from the ansatz
3. the ansatz only determines which linear combination of features to use
4. this is exactly analogous to classical kernel methods: nonlinear in input space, linear in feature space

Intuition. the feature map does all the heavy lifting. the ansatz is just choosing a direction in feature space. making the ansatz more complex (deeper, more parameters) gives u more “directions” to choose from, but the classifier is still fundamentally linear in the quantum features.

7.4 Proof Sketch

claim: for any VQC with feature map U_{ϕ} , ansatz $W(\theta)$, and observable O , the model output $f(x; \theta) = \text{Tr}[\tilde{O}(\theta) \rho(x)]$ is linear in the density matrix features.

proof:

1. the trace is a linear operation: $\text{Tr}[A(\alpha B + \beta C)] = \alpha \text{Tr}[AB] + \beta \text{Tr}[AC]$
2. $\tilde{O}(\theta)$ is a fixed operator for given θ
3. therefore $f(x; \theta) = \text{Tr}[\tilde{O}(\theta) \rho(x)]$ is linear in $\rho(x)$
4. since $\rho(x) = |\phi(x)\rangle \langle \phi(x)|$, the features $\phi_{\alpha}(x) = \text{Tr}[P_{\alpha} \rho(x)]$ span the feature space
5. so f is a linear function of the features ϕ_{α} □

7.5 Connection to Kernel Methods

this directly connects VQCs to quantum kernels:

the quantum kernel is:

$$\kappa(x, x') = \text{Tr}[\rho(x) \rho(x')] = |\langle \phi(x) | \phi(x') \rangle|^2$$

since the VQC is linear in $\rho(x)$, and the kernel is the inner product $\text{Tr}[\rho(x) \rho(x')]$ in the density matrix space, the VQC is doing exactly what a kernel SVM does: linear classification in a high-dimensional feature space, with the kernel encoding the inner products.

Key Point. the punchline: $VQC \equiv$ kernel method. training a VQC is equivalent to finding a linear classifier in the quantum kernel feature space. this means all the theory of kernel methods (generalization bounds, representer theorem, etc.) applies directly to VQCs.

8 When Does the Variational Approach Help?

8.1 VQC vs Direct Kernel Method

if VQCs r just kernel methods in disguise, why not use the kernel directly?

advantages of VQC (variational) approach:

- scales better with dataset size: $O(N)$ circuit evaluations per epoch vs $O(N^2)$ for the kernel matrix
- natural for online learning (can update with one sample at a time)
- can learn feature-dependent weights (if using data re-uploading)
- easier to implement on near-term hardware (one circuit per sample)

advantages of kernel approach:

- convex optimization (no local minima)
- well-understood generalization theory
- no barren plateaus (the kernel is computed, not optimized)
- provably finds the optimal linear classifier in feature space

8.2 When VQC Goes Beyond Kernel

the linear equivalence holds when the ansatz is applied AFTER the encoding. but there r ways to break it:

data re-uploading (Pérez-Salinas et al. 2020):

$$U(x, \theta) = \prod_{\ell} W(\theta_{\ell}) U_{\phi}(x)$$

interleave encoding and trainable layers. this creates a nonlinear dependence on x in the effective observable, going beyond the kernel framework.

adaptive encoding:

$$U_{\phi}(x, \theta_{\text{enc}}) = \text{encoding that depends on trainable parameters}$$

the feature map itself is learned. this is like learning the kernel, which goes beyond fixed-kernel methods.

9 Limitations of the Variational Approach

9.1 Local Minima

the VQC loss landscape is generally non-convex. gradient descent can get stuck in local minima that r far from the global optimum.

Example 9.1. for a 4-qubit VQC with 20 parameters, the loss landscape has many local minima regions. random initialization means u start in a random one and may never find the global minimum. in contrast, the kernel SVM finds the global optimum (convex problem).

9.2 Shot Noise

on real hardware, expectation values r estimated from finite measurements:

$$\langle Z_0 \rangle \approx \frac{1}{S} \sum_{s=1}^S z_0^{(s)} \quad \text{where } z_0^{(s)} \in \{-1, +1\}$$

the variance is:

$$\text{Var}[\hat{y}] = \frac{1 - \langle Z_0 \rangle^2}{S}$$

for $S = 1000$ shots: standard deviation ~ 0.03 . this noise propagates to gradients:

$$\text{Var} \left[\frac{\partial \hat{y}}{\partial \theta_k} \right] \sim \frac{1}{S}$$

noisy gradients slow convergence and can cause oscillations near the optimum.

9.3 Circuit Depth

deeper circuits = more noise on real hardware. for current devices with 2-qubit gate error $p \sim 10^{-3}$ to 10^{-2} :

$$\text{circuit fidelity} \approx (1 - p)^{N_{2Q \text{ gates}}}$$

a VQC with 10 qubits, 3 ansatz layers, and ZZFeatureMap might need ~ 100 two-qubit gates. at $p = 5 \times 10^{-3}$:

$$F \approx (0.995)^{100} \approx 0.61$$

thats 39% noise floor. the classifier is fighting a losing battle.

9.4 Barren Plateaus (Preview)

as well see in lecture 16, random deep circuits have exponentially vanishing gradients:

$$\text{Var} \left[\frac{\partial \mathcal{L}}{\partial \theta_k} \right] \leq O(2^{-n})$$

for 20 qubits: gradient variance $\sim 10^{-6}$. u need $\sim 10^{12}$ shots to detect these gradients above shot noise. completely impractical.

Key Point. the three horsemen of VQC doom: local minima, shot noise, and barren plateaus. all three get worse as u scale up. this is why the kernel perspective is so valuable — it lets u analyze VQCs without actually doing the optimization.

10 Practical Considerations

10.1 Choosing the Feature Map

feature map	entanglement	depth	when to use
ZFeatureMap	none	$O(1)$	simple datasets, fast
ZZFeatureMap	pairwise	$O(n^2)$	feature interactions matter
PauliFeatureMap	custom	varies	u know the problem structure
angle encoding	none	$O(1)$	baseline, always try first

10.2 Choosing the Ansatz

ansatz	params	entangling gates	when to use
RealAmplitudes	$n(L+1)$	$L(n-1)$	default choice
EfficientSU2	$2n(L+1)$	$L(n-1)$	need more expressibility
TwoLocal	custom	custom	hardware-specific design

10.3 Choosing the Optimizer

rule of thumb:

- simulator: L-BFGS-B or Adam (exact gradients, fast convergence)
- real hardware: SPSA (handles shot noise, 2 circuits per step) or COBYLA (gradient-free)
- large parameter count: SPSA (gradient cost independent of p)

10.4 Hyperparameter Guidelines

- number of qubits n : set to input dimension d (or use PCA to reduce d to n)
- ansatz depth L : start with 1–2 layers, increase only if underfitting
- feature map reps r : 1–2 is usually enough
- shots per evaluation: 1024 for prototyping, 4096–8192 for final results
- learning rate: 10^{-2} to 10^{-1} for SPSA, 10^{-3} for Adam
- max iterations: 100–500 (monitor loss, stop early if converged)

11 Example: VQC on Iris Dataset

11.1 Setup

the iris dataset: 150 samples, 4 features, 3 classes. we'll do binary classification on the first 2 classes (setosa vs versicolor, 100 samples).

preprocessing:

1. take first 2 classes (100 samples)

2. standardize features to zero mean, unit variance
3. scale to $[0, \pi]$ for angle encoding
4. 80/20 train/test split

VQC configuration:

- 4 qubits (one per feature)
- ZZFeatureMap, 1 rep
- RealAmplitudes ansatz, 2 layers
- $O = Z_0$ measurement
- COBYLA optimizer, 200 iterations
- 1024 shots (simulated)

11.2 Expected Results

typical performance on iris (2-class):

- train accuracy: 95–100%
- test accuracy: 90–100%
- convergence: ~ 50 –100 iterations
- total circuit evaluations: $\sim 100 \times 100 = 10,000$ (iterations $\times$ training samples)

for comparison, a classical SVM with RBF kernel gets 100% on this same task. the point isnt to beat classical here — iris is tiny. the point is to verify the pipeline works.

11.3 The Qiskit Code Flow

the qiskit implementation flow (describing, not showing code):

1. create the feature map: `ZZFeatureMap(feature_dimension=4, reps=1)`
2. create the ansatz: `RealAmplitudes(num_qubits=4, reps=2)`
3. combine into full circuit: feature map + ansatz
4. create the classifier: `VQC(feature_map, ansatz, optimizer, ...)`
5. fit: `vqc.fit(X_train, y_train)`
6. predict: `vqc.predict(X_test)`
7. score: `vqc.score(X_test, y_test)`

12 Summary

key takeaways from this lecture:

1. the VQC has three stages: feature map $\rightarrow$ ansatz $\rightarrow$ measurement
2. the feature map encodes data into quantum hilbert space; the ansatz provides trainable parameters
3. the full VQC output is $f(x; \theta) = \text{Tr}[\tilde{O}(\theta)\rho(x)]$ — a linear function of the density matrix
4. this means VQCs r linear classifiers in quantum feature space, equivalent to kernel methods
5. nonlinearity comes from the feature map, not the ansatz
6. cost functions: MSE, cross-entropy, hinge loss — all adapted for quantum outputs
7. optimization loop: quantum circuit evaluation + classical parameter update
8. parameter shift rule gives exact gradients; SPSA is practical for noisy hardware
9. limitations: local minima, shot noise, circuit depth, barren plateaus
10. data re-uploading can break the linear equivalence, going beyond kernel methods

the variational-to-linear connection is one of the most important results in QML theory. it tells us that the feature map is king — if u want a better VQC, focus on the encoding, not the ansatz.

next lecture: we make the kernel connection explicit and develop quantum feature space theory from the ground up.